

1. Minimum Spanning trees and the spanning tree polytope

In this document we:

1. introduce the Minimum Spanning Tree (MST) Problem
2. recall and prove the main structural results about MSTs (the Cycle Lemma and the Cut Lemma) and illustrate their role both in obtaining efficient greedy algorithms and as certifying languages
3. describe Kruskal Algorithm
4. strive to identify valid constraints for the spanning tree polytope, in the hope to get a complete description of it in terms of valid constraints
5. get to the standard description of the spanning tree polytope and prove its completeness. The proof is based on Kruskal algorithm and complementary slackness

1.1. preliminary notions

graphs: A graph G is a pair (V, E) comprising a set V of $n := |V|$ elements called *nodes* and a set E of $m := |E|$ elements called *edges*. Every edge $e \in E$ is an unordered pair of nodes in V (*unordered* in that our graphs are undirected, i.e., every edge can be traversed in both directions hence the edge makes no distinction between its two *endnodes*). The *degree* of a node is the number of edges it belongs to. Let $G = (V, E)$ be a graph. Its node and edge sets may be denoted as $V(G) := V$ and $E(G) := E$ respectively. Also, when F is any set of edges, then $V(F)$ is the set of those nodes that are endpoint of at least one edge in F , and we may also regard F as the graph $(V(F), F)$ rather than just an edge set; do it at your preference since multiple viewpoints are always possible and convenient when dealing with graphs.

paths, cycles, connectedness: An u, v -*path* is a graph in which every node has degree 2 except u and v , both of which have degree 1, and where it is also possible to move from u to v in precisely $n - 1$ steps, where every step moves from one endnode to the other of an edge of the path. All $m = n - 1$ edges of the path get traversed when we move from u to v in this way. The *length* of the path is m . A graph (V, E) is *connected* if it contains (see the concept of subgraph more below) an u, v -path for every two nodes $u, v \in V$. A *cycle* is what we obtain when to an u, v -path we add an edge with endnodes u and v . Cycles have $m = n$ edges and remain connected (actually, become a path) whatever edge we remove from them (a single edge deletion).

subgraphs, spanning and induced, cuts: A graph $H = (U, F)$ is a *subgraph* of a graph $G = (V, E)$ when $U \subseteq V$ and $F \subseteq E$; the subgraph H is said to *span* G if $V(F) = V$. A graph (V, E) admits a spanning subgraph iff it has no *isolated nodes*, i.e., iff $V(E) = V$. Where S is any subset of V , then $\langle S \rangle$ denotes the set of those edges in E having both nodes in S and the subgraph $(S, \langle S \rangle)$ is called *the subgraph of G induced by S* . When both S and $V \setminus S$ are non-empty, then the *cut* of shores S and $V \setminus S$ is the set $\delta(S)$ of those edges in E having one node in S and the other in $V \setminus S$.

forests are made of trees: A *forest* is an acyclic graph, or a set E of edges such that $(V(E), E)$ is a forest. A *tree* is a connected forest. In every tree, $|E| = |V| - 1$ and every two nodes u, v are connected by a unique path. Whenever an edge $uv \notin T$ is added to a tree T , this path together with the edge uv will form a cycle; this is the sole cycle in $T + uv$ and is called the *fundamental cycle of uv w.r.t. T* . A forest (or tree) of a graph G is any forest (or tree) which is a subgraph G . Prove yourself that a graph (V, E) admits a spanning tree iff no cut $\delta(S)$ of G is empty.

loops and parallel edges, simple graphs: A *loop* of a graph is any edge whose two endnodes are actually the same node. Two non-loop edges e and f of a graph are said to be *parallel* if their unordered pair of endnodes is the same. A graph is more properly called a *multi-graph* as soon as it

has loops or parallel edges, otherwise it is said to be *simple*. Given our interest into the MST Problem, we could restrict our attention to graphs with no loops, since no loop can be part of a tree nor of a forest. We however prefer to allow for loops, so that the operations of contraction can be defined in simplicity without leading us outside of the class of instances we allow. Similarly, we could always ensure that the input instance comprises a graph with no parallel edges since we could simply preprocess our instance as follows: out of a bunch of parallel edges, we retain only one that has minimum cost and delete all the others. Again, we find more convenient to allow for parallel edges as well, again to prevent contraction to lead us outside the class of intended instances, and also for a neat understanding of the class of spanning trees containing a specific edge. Of course, when the graph is simple, a lighter and more convenient notation is possible as each edge e is univocally identified by its two endnodes u and v ; to denote it we may write $\{u, v\}$ (or, more shortly, uv) rather than e . We denote by K_n the *complete simple graph* on n nodes, *complete* means that all possible $\binom{n}{2} := \frac{n(n-1)}{2}$ edges are present; in K_n the degree of every node is $n - 1$. When dealing with K_n , we will profit of this convenient notation.

deletion and contraction: When in the graph (V, E) we *delete* an edge $e \in E$, then we obtain the graph $G \setminus e := (V, E \setminus e)$. When in the graph (V, E) we *contract* an edge e of endnodes u and v , then we obtain the graph $G \setminus e$ by first deleting edge e and then identifying nodes u and v in a single new node v_e (we remark that, in this second step, for every edge in $E \setminus e$ every endpoint which is either u or v gets replaced by v_e). Contracting a loop amounts to deleting it. Note that, in this way, every edge of G which was parallel to e becomes a loop in $G \setminus e$, while any two edges e_u and e_v of G having a common endnode in $z \neq u, v$ but the other endnode in u for e_u and in v for e_v , become parallel in $G \setminus e$.

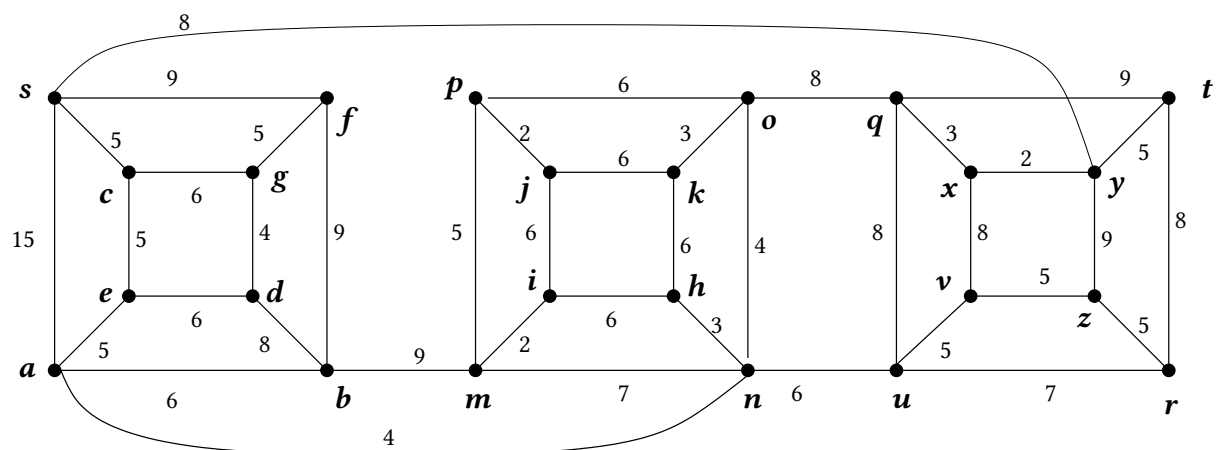
1.2. The minimum spanning tree problem

The minimum spanning tree problem (MSTP) asks to find a minimum spanning tree (MST) in a graph with costs on its edges.

INPUT: an undirected graph $G = (V, E)$ with costs $c : E \rightarrow \mathbb{R}_+$ on its edges.

OUTPUT: a spanning tree T of G minimizing the functional $c(T) := \sum_{e \in T} c_e$.

Example: if the input is the weighted graph depicted in this figure:

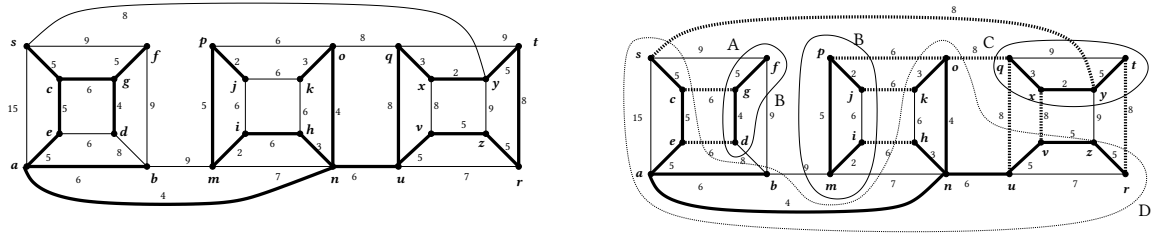


Then a possible MST is given in the following figure on the left (it comprises the edges drawn in bold line). On the right, all edges are classified in three categories:

solid thick line those belonging to each MST

thick dotted line those who belong to some STDs but not all

thin line those who do not belong to any STD



Here, the edges drawn in thick line are those belonging to every MST and gather together the nodes in 4 islands A , B , C and D .

Every MST must take at least one edge from the cut that separated the component A from the rest of the graph. Every such arc has a node in A and the other in D , hence we are due to choose precisely one of these edges, knowing that they are interchangeable in order to obtain a feasible solution. We hence take any edge of minimum cost (6). Here we have two options: either cg or ed . Analogously, to connect component B to the rest of the graph we are due to take precisely one edge with a node in B and the other outside (i.e., an edge of the cut $\delta(B)$), since taking more than one such edge would otherwise result in forming a cycle and would not help in ensuring connectivity. Here the options are 3 and this choice is completely independent from the previous one. Finally, in order to connect component C to the block $D + A + B$ we must choose one edge of cost 8 (here there are 5 options).

The total number of MSTs is therefore $2 \times 3 \times 5 = 30$.

1.3. Fundamental facts and lemmas

Fact [edge deletion]: The spanning trees of $G = (V, E)$ that do not include a specific edge $e \in E$ are in one-to-one correspondence with the spanning trees of the graph $(V, E \setminus e)$ obtained from G by *deleting* edge e .

Fact [edge contraction]: The spanning trees of $G = (V, E)$ that do include a specific edge $e = uv \in E$ are in one-to-one correspondence with the spanning trees of the multi-graph $G \setminus e$ obtained from G by *contracting* edge e .

Fact 0: if C is a cycle and $\delta(S)$ is a cut then the number of edges belonging to both is even.

Lemma [Cycle]: If e is an edge of maximum cost in some cycle C of G then there exists an MST T of G with $e \notin T$.

Lemma [Cut]: If e is an edge of minimum cost in some cut $\delta(S)$ of G then there exists an MST T of G with $e \in T$.

These two lemmas provide us with two convenient languages to yes-certificate or no-certificate on whether an edge belongs to some/all/no MSTs for the given input. For example, still with reference to the graph pictured more above:

edge mb in no MST since it is the edge of strictly maximum cost in the cycle $mnurab$.

edge mp in all the MSTs since it is the edge of strictly minimum cost in the cut $\delta(\{m, i\})$.

edge mn in some MST since it is an edge of minimum cost in the cut that separates the component D from the rest of the graph. Not in all the MSTs since it is an edge of maximum cost in the cycle $mnop$.

proof of the Cycle Lemma: Let T be any MST. If $e \notin T$ then we are done. Otherwise, let C_1 and C_2 be the two connected components of the forest $F := T \setminus e$. Note that the edges e' of G for which $F \cup \{e'\}$ is again a tree are precisely those of the cut $\delta(C_1)$. Since $e \in \delta(C_1) \cap C$ then there must exist another edge f in $\delta(C_1) \cap C$ by Fact 0. Since e is an edge of maximum cost in the cycle C than the cost of the tree $F \cup \{f\}$ does not exceed the cost of T . QED

There is a duality between the statements of the Cycle Lemma and the Cut Lemma. In fact, the proof of the Cut Lemma can be read out from that of the Cycle Lemma, through a sort of duality mirror.

proof of the Cut Lemma: Let T be any MST. If $e \in T$ then we are done. Otherwise, let C be the cycle in $T \cup e$. Note that the edges e' of G for which $T \cup \{e\} \setminus e'$ is again a tree are precisely those of the cycle C . Since $e \in C \cap \delta(S)$ then there must exist another edge f in $C \cap \delta(S)$ by Fact 0. Since e is an edge of minimum cost in the cut $\delta(S)$ than the cost of the tree $T \cup \{e\} \setminus f$ does not exceed the cost of T . QED

1.4. Kruskal Algorithm

Here is a pseudocode description of the Kruskal Algorithm for the computation of an MST in a given connected graph G with weights on its edges:

Function $\text{Kruskal}(G, c)$:

1. let $e_1, e_2, \dots, e_m$ be any ordering of the edges in $E(G)$ by non-decreasing cost
2. $F_0 := \emptyset; G_0 := G$
3. for $i = 1, 2, \dots, m$:
4. if $F_{i-1} \cup \{e_i\}$ contains a cycle then $G_i := G_{i-1} \setminus e_i$ and $F_i := F_{i-1}$
5. else $F_i := F_{i-1} \cup \{e_i\}$;
6. return F_m

Clearly, the edge set F_m returned by the call $\text{Kruskal}(G, c)$ is a forest of G since the acyclicity of F_i is an invariant of the “for i cycle” issued at line 4. Moreover, if G is connected then F_m will be connected and spanning as well (since $F_m \cap \delta(S) \neq \emptyset$ for every cut $\delta(S)$ of G , as the very first edge e_i of $\delta(S)$ in the order $e_1, e_2, \dots, e_m$ will have been taken into F_i). Hence F_m is indeed a spanning tree. This could have been more neatly deduced also by considering just the following three invariants:

1. F_i is a forest for every i
2. G_i is connected for every i
3. $E(G_i) \cup F_i = E(G)$ for every i

and observing that Invariant 3 implies $F_m = G_m$.

The fact that the spanning tree F_m is an MST can be deduced from the Cycle Lemma with this consideration: since whenever a cycle C was present in $F_{i-1} \cup \{e_i\}$ then e_i was an edge of maximum cost in the cycle C of the graph G_{i-1} , hence it can be deleted away from G_{i-1} , which results in moving to G_i at this point.

The optimality of the spanning tree F_m can be also deduced from the Cut Lemma, reasoning now as follows: whenever an edge $e_i = uv$ gets added to F_{i-1} to get F_i then e_i is an edge of minimum cost in the cut $\delta(S)$ of the graph G_{i-1} , where S is the connected component of F_{i-1} containing node u (note that S does not contain v for otherwise $F_{i-1} \cup \{e_i\}$ would contain a cycle).

Only to illustrate the conceptual power of the dual operations of deletion and contraction, we next offer a more succinct pseudocode description of the Kruskal Algorithm in terms of these two dual operations, which have played a fundamental role in the development of Graph Theory:

Function $\text{Kruskal_del_con}(G, c)$:

1. $T := \emptyset$
2. while G has some edges:
3. let e be any edge of G of minimum cost
4. $T := T \cup \{e\}; G := G \setminus e$
5. return T

1.5. The spanning tree polytope

The spanning tree polytope for a given graph $G = (V, E)$ is the convex hull of the characteristic vectors $x_T : E \rightarrow \{0, 1\}$ of the spanning trees T of G .

For example, for the graph $K_3 = (\{a, b, c\}, \{ab, ac, bc\})$, we have three spanning trees:

1. $T_1 = \{ab, ac\}$, with characteristic vector $\chi_1 = (1, 1, 0)$
2. $T_2 = \{ab, bc\}$, with characteristic vector $\chi_2 = (1, 0, 1)$
3. $T_3 = \{ac, bc\}$, with characteristic vector $\chi_3 = (0, 1, 1)$

and the spanning tree polytope would be the convex set $\text{conv_hull}(\text{ST}; K_3) = \{\lambda_1\chi_1 + \lambda_2\chi_2 + \lambda_3\chi_3 : \lambda_1 + \lambda_2 + \lambda_3 = 1, \lambda_1, \lambda_2, \lambda_3 \geq 0\}$.

However, what polyhedral combinatorics is really interested into is the description of $\text{conv_hull}(\text{ST}; K_3)$ in terms of linear constraints, that is:

$$\text{LP}(\text{ST}; K_3) = \{(x_{ab}, x_{ac}, x_{bc}) \in \mathbb{R}^3 : x_{ab} + x_{ac} + x_{bc} = 2, 0 \leq x_{ab}, x_{ac}, x_{bc} \leq 1\}.$$

In this second description we discover that the polytope $\text{conv_hull}(\text{ST}; K_3)$ is contained in the plane $x_{ab} + x_{ac} + x_{bc} = 2$, due to the fact that every tree on 3 nodes has precisely 2 edges. This plane Π has affine dimension 2 since every set of points of Π that *spans* Π , (i.e., whose convex hull is a full dimensional polytope in Π) has at least 3 vertices. All other constraints of the description (the three non-negativity constraints and the three constraints upper bounding to 1 each variable) are also valid inequalities since every vertex of $\text{conv_hull}(\text{ST}; K_3)$ (that is, every characteristic vector of a spanning tree of K_3) satisfies all six of them. In fact, they are *facet defining inequalities* in that, for each of them, the affine dimension of the intersection of Π with the plane described by the inequality read as an equation is 1, i.e., the affine dimension of Π reduced exactly by 1. By now we have assessed that $\text{conv_hull}(\text{ST}; K_3) \subseteq \text{LP}(\text{ST}; K_3)$, by the validity of each constraint imposed by the linear system of $\text{LP}(\text{ST}; K_3)$.

We next argue that $\text{LP}(\text{ST}; K_3)$ offers a *complete description* $\text{conv_hull}(\text{ST}; K_3)$, that is, it misses no facet of the polytope $\text{conv_hull}(\text{ST}; K_3)$. In other words, the two descriptions deliver two different views of the very same polytope. To show this, since we already know that $\text{conv_hull}(\text{ST}; K_3) \subseteq \text{LP}(\text{ST}; K_3)$, it suffices to show that every vertex of $\text{LP}(\text{ST}; K_3)$ is also a vertex of $\text{conv_hull}(\text{ST}; K_3)$. Indeed, every vertex of the polytope $\text{LP}(\text{ST}; K_3)$ corresponds to a basic solution of its linear system. Since the affine dimension of $\text{conv_hull}(\text{ST}; K_3)$ is 2, then every basic solution is characterized by 2 linearly independent constraints satisfied at equality. Then consider:

1. when two or more non-negativity constraints are tightened then we obtain a non-feasible system

2. when two upper bounds constraints are tightened (say, imposing $x_{ab} = 1$, and $x_{bc} = 1$) then the unique solution to the tightened constraints system is indeed a vertex of $\text{conv_hull}(\text{ST}; K_3)$ and it so happens that also one single non-negativity constraint is satisfied with equality (say, $x_{ac} = 0$)

As a consequence, no other cases need to be considered in order to conclude that $\text{conv_hull}(\text{ST}; K_3) = \text{LP}(\text{ST}; K_3)$. The minimal constraints description is no unique however, since

$$\text{LP}'(\text{ST}; K_3) = \left\{ (x_{ab}, x_{ac}, x_{bc}) \in \mathbb{R}^3 : x_{ab} + x_{ac} + x_{bc} = 2, x_{ab}, x_{ac}, x_{\{b,c\}} \geq 0, x_{ab}, x_{ac} \leq 1, x_{ab} + x_{ac} \geq 1 \right\}.$$

is also a complete constraints description. Indeed, the extra constraint $x_{ab} + x_{ac} \geq 1$ is satisfied by all three vertices χ_1 , χ_2 and χ_3 of $\text{conv_hull}(\text{ST}; K_3) = \text{LP}(\text{ST}; K_3)$, while the missing constraint $x_{bc} \leq 1$ can be deduced by combining the extra constraint with the equality constraint $x_{ab} + x_{ac} + x_{bc} = 2$.

Note that MSTP is one of those optimization problems on weighted graphs that can be solved for a generic graph G as soon as they can be solved in the special case when $G = K_n$ for some $n \in \mathbb{N}$ (just add all the missing edges associating a huge cost to each of them). As a matter of fact, a constraints description for $\text{conv_hull}(\text{ST}; G)$ can be readily obtained from a constraints description for $\text{conv_hull}(\text{ST}; K_n)$, with $n = |V(G)|$, by adding the constraint $x_e = 0$ for every edge $e \in E(K_n) \setminus E(G)$. As for multi-graphs, then assuming that E_{uv} is the set of edges with one endpoint in u and the other in v , then the idea would be to replace each occurrence of the variable x_{uv} in the objective function or in any linear constraint with the expression $\left(\sum_{e \in E_{uv}} x_e \right)$.

The interest in having a complete descriptions in terms of valid constraints is high and practical, since it allows to solve MSTP in polynomial time by resorting on the theory of Linear Programming. The standard challenge in polyhedral combinatorics is therefore to make sense of the constraints in the second description as for graphs on every possible number of nodes. Making sense in at least one of these two possible ways:

1. **recognition of validity:** obtaining a polynomial time algorithm that, when given $n \in \mathbb{N}$, and a linear constraint $C(x)$ on the variables associated to the edges of K_n , checks whether $C(x)$ is *valid* (i.e., respected by every point in $\text{conv_hull}(\text{ST}; K_n)$) and, in case not, returns a vertex of $\text{conv_hull}(\text{ST}; K_n)$ (i.e., the characteristic vector χ_T of some spanning tree T of K_n) that violates $C(x)$.
2. **separation:** obtaining a polynomial time algorithm (called the *separating oracle*) that, when given $n \in \mathbb{N}$, and a point $x \in \mathbb{R}^{\binom{n}{2}}$, tells whether $x \in \text{conv_hull}(\text{ST}; K_n)$ and, in case not, returns any valid constraint for $\text{conv_hull}(\text{ST}; K_n)$ which is however violated by x .

Therefore, if we could act *recognition of validity* in polynomial time, then MSTP would be in $\text{NP} \cap \text{coNP}$ by the Strong Duality Theorem of Linear Programming. It is also known that, as a consequence of the ellipsoid algorithm, separation=optimization, that is, if we could act *separation* in polynomial time, then the optimization MSTP would be solvable in polynomial time as well, even though the number of constraints in a minimal constraint description for $\text{conv_hull}(\text{ST}; K_n)$ grows exponentially in n . (Note: in order for the ellipsoid algorithm to run in polynomial time in the size of the input the valid constraint returned is not required to be facet defining, as long as it is returned in polynomial time in the size of each query.)

The convex hull of a finite set of points is always a polytope; when it is fully dimensional then its minimal constraint description is always unique: it contains precisely those inequalities that are facet defining. The polytope $\text{conv_hull}(\text{ST}; K_n)$ is not fully dimensional since it is contained in the

plane $\sum_{e \in E} x_e = n - 1$, however, $\sum_{e \in E} x_e = n - 1$ is essentially (i.e., except for multiplying if by a non-zero real number) the only linear equation that is valid for $\text{conv_hull}(\text{ST}; K_n)$, hence we can take $\sum_{e \in E} x_e = n - 1$ as our first constraint. After this, since $\text{conv_hull}(\text{ST}; K_n)$ is full dimensional within the plane $\sum_{e \in E} x_e = n - 1$, then every other constraint in any minimal description of $\text{conv_hull}(\text{ST}; K_n)$ will be a facet defining inequality. Indeed, the affine dimension of the plane $\sum_{e \in E} x_e = n - 1$ can not be bigger than $m - 1 := \binom{n}{2}$ since the plane is contained in $\mathbb{R}^m$. To show that it is precisely $m - 1$, we exhibit m affinely independent points $p_e, e \in E(K_n)$; here p_e is the vector such that $p_{e(e)} = n - 1$ and $p_{e(e')} = 0$ for every $e' \in E \setminus \{e\}$. To show that $\text{conv_hull}(\text{ST}; K_n)$ is full dimensional within the plane $\sum_{e \in E} x_e = n - 1$ we need to exhibit m affinely independent points of $\text{conv_hull}(\text{ST}; K_n)$. The natural thing to do is to search among the characteristic vectors of spanning trees. Let H be any Hamiltonian cycle of K_n . For the first n points choose any of the n edges of H and consider the spanning tree obtained from H by removing that edge. For the remaining $m - n$ points consider any edge $e \in E \setminus H$ and choose a spanning tree T_e of $H \cup \{e\}$ with $e \in T_e$. It is easy to check that these $n + (m - n) = m$ spanning trees are affinely independent.

The natural challenge at this point is to identify families of constraints that offer a complete description of $\text{conv_hull}(\text{ST}; K_n)$ for a generic $n \in \mathbb{N}$. Not only to get hold on a complete set of them, but also to make some sense of them via some appealing combinatorial interpretation. For example, inspired by the Cut Lemma one could argue that every spanning tree should intersect every cut, hence write the following general family of valid constraints:

$$x(\delta(S)) \geq 1 \text{ for every } S \subsetneq V \text{ with } S \neq \emptyset$$

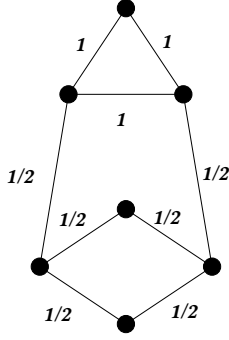
This family of constraints is appealing. For the special case $n = 4$, when combined with the non-negativity and upper bound constraint on each single variable and the sole equation constrain offers a complete description of $\text{conv_hull}(\text{ST}; K_4)$, that is:

$$\text{conv_hull}(\text{ST}; K_4) = \{(x_{ab}, x_{ac}, x_{ad}, x_{bc}, x_{bd}, x_{cd}) \in \mathbb{R}^6 : x_{ab} + x_{ac} + x_{ad} + x_{bc} + x_{bd} + x_{cd} = 3, 0 \leq x_{ab}, x_{ac}, x_{ad}, x_{bc}, x_{bd}, x_{cd} \leq 1, x_{ab} + x_{ac} + x_{ad} \geq 1, x_{ab} + x_{bc} + x_{bd} \geq 1, x_{ac} + x_{bc} + x_{cd} \geq 1, x_{ad} + x_{bd} + x_{cd} \geq 1\}.$$

Here above we have actually omitted the constraints of the form $x(\delta(S)) \geq 1$ for those $S \subset V$ with $|S| = 2$, like $x_{ac} + x_{ad} + x_{bc} + x_{bd} \geq 1$, since each of them can be derived by combining the equation $x_{ab} + x_{ac} + x_{ad} + x_{bc} + x_{bd} + x_{cd} = 3$ with two upper bound constraints. For example, $x_{ac} + x_{ad} + x_{bc} + x_{bd} \geq 1$ follows from $x_{ab} + x_{ac} + x_{ad} + x_{bc} + x_{bd} + x_{cd} = 3$ combined with $x_{ab} \leq 1$ and $x_{cd} \leq 1$. We can here understand why there is no unique minimal constraints description of a polytope which is not full dimensional: we could have dropped instead the inequality $x_{ab} \leq 1$, which would have been implied by $x_{ab} + x_{ac} + x_{ad} + x_{bc} + x_{bd} + x_{cd} = 3$ and $x_{cd} \leq 1$.

One could hence suspect at this point that the family of constraints $x(\delta(S)) \geq 1$ is actually more than enough. However, on the contrary, redundancy is not necessarily a good sign since it could on the contrary reveal that our inelegance comes from being still missing something.

Indeed, the following fractional point reveals the incompleteness of the families of constraints proposed above already for $n = 6$:



The reader can readily check that the above fractional point satisfies all of the constraints seen until here, while, clearly, it can not belong to $\text{conv_hull}(\text{ST}; K_6)$ since the three edges of value 1 form a cycle C , while $x(C) \leq |C| - 1$ should be a valid inequality for $\text{conv_hull}(\text{ST}; K_6)$ since no tree contains cycles.

More in general, for every $n \in \mathbb{N}$ and for every cycle C of K_n , $x(C) \leq |C| - 1$ should be a valid inequality for $\text{conv_hull}(\text{ST}; K_n)$. This brings our mind back to the Cycle Lemma, it appears we should use also that one. Note that the cycles are as many as $(n - 1)!$ whereas the cuts are only $2^{n-1} - 2$. Many questions and doubts could arise at this point: maybe the cycles are enough and we do not need the cuts? Is the order in which the nodes appear in the cut really relevant? (the order of the nodes is what makes the cycles being so many)

At this point one could realize that these cycle constraints are dominated by stronger constraints of the form:

$$x(< S >) \leq |S| - 1 \text{ for every } S \subseteq V \text{ with } |S| \geq 2$$

Since all variables are non-negative, one such single constraint would strictly dominate each of the $(|S| - 1)!$ cycle constraints associated to the Hamilton cycles of the subgraph of K_n induced by S . Note also that all the upper bound constraints on the single variables are obtained as the special case when $|S| = 2$. Also all the cut constraints are implied by this new family of $2^n - n - 1$ constraints, combined with the equality constraint $\sum_{e \in E} x_e = n - 1$. Indeed, $E = < S > \cup \delta(S) \cup < V \setminus S >$. As such,

$$\begin{aligned} x(\delta(S)) &= x(E) - x(< S >) - x(< V \setminus S >) \geq \\ &\geq (n - 1) - (|S| - 1) - (|V \setminus S| - 1) = n - 1 - |S| + 1 - |V| + |S| + 1 = 1 \end{aligned}$$

We begin to suspect that the following description might be complete for the spanning tree politope:

$$\text{LP}(\text{ST}; K_n) = \{ (x \in \mathbb{R}^{\binom{n}{2}} : x \geq 0, x(E) = n - 1, x(< S >) \leq |S| - 1 \text{ for every } S \subseteq V \text{ with } |S| \geq 2) \}$$

You can find several proofs of this fact on books. In the next section we aim to offer one that might be simpler and reveal the power of Linear Programming Theory and a possible use of the Complementary Slackness Theorem in the joint challenge to prove good conjectures while designing combinatorial algorithms.

1.6. our own proof of completeness based on Kruskal Algorithm

An *integral point* of $\mathbb{R}^m$ is a point that actually belongs to $\mathbb{Z}^m$. An *integral polyhedron* is a polyhedron all of whose vertices are integral. The following simple observation shows that the integrality of $\text{LP}(\text{ST}; G)$ is all what we ought to prove in order to prove that $\text{LP}(\text{ST}; G) = \text{conv_hull}(\text{ST}; G)$, that is, $\text{LP}(\text{ST}; G)$ offers a complete description of the spanning tree polytope of G .

Observation: Every integral point in $\text{LP}(\text{ST}; G)$ is the characteristic vector of a spanning tree of G .

proof: For every $e \in E$, $x_e \geq 0$ by the non-negativity constraints, and $x_e \leq 1$ by the constraint $x(\langle S \rangle) \leq |S| - 1$ where S is the set comprising the two endnodes of e . Hence x is a 0, 1-vector and can be regarded as the characteristic vector of some subset F of E . Note that F is a forest since, if F contained a cycle C , then x would violate the constraint $x(\langle V(C) \rangle) \leq |V(C)| - 1$. Also, $|F| = n - 1$ since $x(E) = n - 1$. Hence F is a spanning tree. QED

Indeed, some proofs in the literature follow this approach, which has developed its own general techniques valid also for other problems. In the case of MSTP these proofs are not that simple however, requiring the technique of uncrossing, and with the equality constraint adding to the general clutter.

To offer a proof as intuitive as possible, we directly prove that every vertex of $\text{LP}(\text{ST}; G)$ is the characteristic vector of a spanning tree. Clearly, this is achieved if we prove that for every pair (G, c) there exists an optimal solution to the following LP which is the characteristic vector of some spanning tree of G .

$$\begin{aligned} & - \max \sum_{e \in E} -c_e x_e \\ & \sum_{e \in E} x_e = n - 1 \\ & \sum_{e \in \langle S \rangle} x_e \leq |S| - 1 \text{ for every } S \subseteq V \text{ with } |S| \geq 2 \\ & x_e \geq 0 \text{ for all } e \in E \end{aligned}$$

The dual of the above LP is:

$$\begin{aligned} & - \min (n - 1)y_{=} + \sum_S (|S| - 1)y_S \\ & y_{=} + \sum_{S: e \in \langle S \rangle} y_S \geq -c_e \quad \text{for every } e \in E \\ & y_S \geq 0 \quad \text{for every } S \subseteq V \text{ with } |S| \geq 2 \\ & y_{=} \text{ free} \end{aligned}$$

To implement our plan, we extend Kruskal Algorithm so that, on every input pair (G, c) , it returns not only a spanning tree T of G but also a proof of its optimality expressed in terms of a feasible solution of Problem D that satisfies the complementary slackness conditions w.r.t. χ_T .

Function $\text{Kruskal_Extended}(G, c)$:

1. let $e_1, e_2, \dots, e_m$ be any ordering of the edges in $E(G)$ by non-increasing value
2. $F_0 := \emptyset$; $x_e = 0$ for all e ; $y_{=} = \max_{e \in E} -c_e$; $y_S = 0$ for all S , $S = V$
3. for $i = 1, 2, \dots, m$:
4. if $F_{i-1} \cup \{e_i\}$ contains a cycle then:
5. $F_i := F_{i-1}$
6. else:
7. $F_i := F_{i-1} \cup \{e_i\}$; $x_{e_i} = 1$
8. let $S_i \subseteq V$ be the connected component of F_i containing e_i

9. $y_{S_i} := -c(e_i) - y_-; \mathbb{S} := \mathbb{S} \cup \{S_i\}$
10. for each of the at most two $S' \in \mathbb{S}$ strictly contained in S :
11. $\mathbb{S} := \mathbb{S} \setminus \{S'\}; y_{S'} := y_{S'} - c(e_i)$
12. return F, x, y

We invite the reader to check the following invariants of the above algorithm:

1. F is a forest at every iteration of the algorithm, and in the end it is a spanning tree (we proved this already)
2. at every iteration of the algorithm, $x = \chi_F$ and, as such, satisfies all primal inequalities; the primal equation is satisfied only since F ultimately becomes a spanning tree of G
3. the dual solution y is feasible at every step of the algorithm
4. at every step of the algorithm the pair (x, y) satisfies the complementary slackness conditions except that on y_- and on the equation imbalance $n - 1 - \sum_{e \in E} x_e$. In the end also this condition is satisfied since the equation $\sum_{e \in E} x_e = n - 1$ is ultimately satisfied.

The first two invariants should be clear, and had already been seen in the classical version of the Kruskal Algorithm. Initially, the dual solution is feasible and it remains such by the way we update it. Initially, the pair (x, y) satisfies the complementary slackness conditions and, whenever an $x(e_i)$ gets set to 1, then the slack variable associated to the corresponding constraint of the dual (namely, $y_- + \sum_{S: e_i \in S} y_S + c(e_i)$) drops to 0 since we set $y_{S_i} = -y_- - c(e_i)$. At the same time, the slack variable associated to the primal constraint $\sum_{e \in S_i} x_e \leq |S_i| - 1$ drops to 0 since with the addition of edge e_i to F we obtain that $|F \cap S_i| = |S_i| - 1$.

1.7. matroids (a more general result for a slightly simpler proof of completeness)

An alternative way to get a clean but longer proof would be to obtain the result as a special case of the characterization of the polytope for the bases of a matroid.

This path has two advantages:

1. you bring home a more general result
2. simplicity, coming from two facts:
 - 2.1. with matroids you are naturally maximizing the objective function, hence the optimization problem is in standard form
 - 2.2. you have no equality constraint, the equality constraint here introduces a confusionary twist

Though a longer journey, this would certainly make for a transparent and didactically worthy proof, and in fact this is what is proposed in many books. These proofs for matroids often involve uncrossing or ranking functions, but one could resort on approach to propose an algorithm based on the complementary slackness conditions that we have proposed here, to keep the proof elementary. If you want a simple and clean proof without having to learn about matroids in general, in the next section we customize the approach for matroids to the special case of graphic matroids, which will ultimately deliver the sought for result.

NOT SURE IT IS WORTH WRITING DOWN (maybe in another life or as a project for a student)

1.7.1. Welcome



to the home page of our course in Discrete Optimization and Decision Making